

KANISHKA SHARMA

CU ID: CU24250095 | Roll No: 26 | Section: B

Full Stack Web Development — Mini E-Commerce Project Lab Sheet 3: Rebuilding with React (Components, Props, useState)

Project: *techmart-react* (Vite + React)

PART: 01 PROJECT SETUP

Created with: `npm create vite@latest techmart-react -- --template react`, then `npm install`.

package.json

```
{
  "name": "techmart-react",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.5",
    "react-dom": "^19.2.5"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.5"
  }
}
```

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
});
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>TechMart</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

PART: 02 MOCK DATA — src/products.js

Product catalog carried over from Lab Sheet 2, exported as a plain array for now (Task 2).

```
// Temporary mock data source.
// Moved over from Lab Sheet 2's script.js product catalog.
const products = [
  { id: 1, name: "Wireless Earbuds",    price: 1999, image: "/images/earbuds_real.png" },
  { id: 2, name: "Smart Watch",         price: 3499, image: "/images/watch_real.png" },
  { id: 3, name: "Bluetooth Speaker",   price: 1299, image: "/images/speaker_real.png" },
  { id: 4, name: "Power Bank",          price: 999,  image: "/images/powerbank_real.png" },
  { id: 5, name: "Mechanical Keyboard", price: 2799, image: "/images/keyboard_real.png" },
  { id: 6, name: "Wireless Mouse",      price: 699,  image: "/images/mouse_real.png" },
  { id: 7, name: "HD Webcam",           price: 1599, image: "/images/webcam_real.png" },
  { id: 8, name: "Laptop Stand",        price: 899,  image: "/images/standd_real.png" },
];

export default products;
```

PART: 03 COMPONENTS

1. src/components/ProductCard.jsx

Receives a single product + onAddToCart via props (Task 3). Purely presentational — holds no state of its own.

```
// ProductCard: a "dumb" presentational component.
// It receives a single product via props and a callback (onAddToCart) via props.
// It holds no cart state itself – state stays lifted up in App.
function ProductCard({ product, onAddToCart }) {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>₹{product.price.toLocaleString("en-IN")}</p>
      <button onClick={() => onAddToCart(product)}>Add to Cart</button>
    </div>
  );
}

export default ProductCard;
```

2. src/components/ProductList.jsx

Imports `products.js`, maps over the array, renders one `ProductCard` per product using `product.id` as the key — never the array index (Task 4).

```
import products from "../products";
import ProductCard from "../ProductCard";

// ProductList imports the mock data, maps over it, and renders one
// ProductCard per product. Uses product.id as the key – NOT the array index –
// so React can correctly track items if the list is ever filtered/sorted.
function ProductList({ onAddToCart }) {
  return (
    <section>
      <h2>All Products</h2>
      <div className="product-grid">
        {products.map((product) => (
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={onAddToCart}
          />
        ))}
      </div>
    </section>
  );
}

export default ProductList;
```

3. src/components/Cart.jsx

Receives the cart array + removeFromCart/updateQuantity via props (Task 6). Shows a "Your cart is empty" message via an early return when `cart.length === 0` (Task 9), otherwise renders the table and grand total.

```
// Cart receives the cart array + mutator functions as props.
// It never mutates cart itself – it only calls back up to App.
function Cart({ cart, removeFromCart, updateQuantity, onCheckout }) {
  // Conditional rendering: empty cart gets its own message (Task 9)
  if (cart.length === 0) {
    return (
      <div className="cart-panel">
        <h2>Your Cart</h2>
        <p className="empty-cart-msg">Your cart is empty.</p>
      </div>
    );
  }

  const grandTotal = cart.reduce(
    (sum, item) => sum + item.product.price * item.quantity,
    0
  );

  return (
    <div className="cart-panel">
      <h2>Your Cart</h2>
      <table>
        <thead>
          <tr>
            <th>Item</th>
            <th>Quantity</th>
            <th>Price</th>
            <th>Subtotal</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          {cart.map((item) => (
            <tr key={item.product.id}>
              <td>{item.product.name}</td>
              <td>
                <input
                  type="number"
                  min="1"
                  className="qty-input"
                  value={item.quantity}
                  onChange={e =>
                    updateQuantity(item.product.id, Number(e.target.value))
                  }
                />
              </td>
              <td>₹{item.product.price.toLocaleString("en-IN")}</td>
              <td>₹{(item.product.price * item.quantity).toLocaleString("en-IN")}</td>
              <td>
                <button
                  className="remove-btn"
                  onClick={() => removeFromCart(item.product.id)}
                >
                  Remove
                </button>
              </td>
            </tr>
          ))}
        </tbody>
        <tfoot>
          <tr className="total-row">
            <td colspan="3">Grand Total</td>
            <td colspan="2">₹{grandTotal.toLocaleString("en-IN")}</td>
          </tr>
        </tfoot>
      </table>
    </div>
  );
}
```

```
        </tr>
      </tfoot>
    </table>
    <div style={{ textAlign: "center" }}>
      <button className="checkout-btn" onClick={onCheckout}>
        Proceed to Checkout
      </button>
    </div>
  </div>
);
}

export default Cart;
```

4. src/components/CheckoutForm.jsx

Controlled inputs throughout — every field's value comes from the form state object and every keystroke goes through `handleChange` via `onChange` (Task 8). Validation mirrors Lab Sheet 2's rules and writes messages into state instead of using `alert()`.

```
import { useState } from "react";

// All inputs are controlled: value comes from state, onChange updates state.
function CheckoutForm({ onOrderPlaced }) {
  const [form, setForm] = useState({
    name: "",
    address: "",
    city: "",
    pincode: "",
    phone: "",
    payment: "cod",
  });
  const [errors, setErrors] = useState({});
  const [placed, setPlaced] = useState(false);

  function handleChange(e) {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  }

  function validate() {
    const newErrors = {};
    if (!form.name.trim()) newErrors.name = "Name is required.";
    if (!form.address.trim()) newErrors.address = "Address is required.";
    if (!/^\\d{6}$/.test(form.pincode.trim()))
      newErrors.pincode = "Pincode must be exactly 6 digits.";
    if (!/^\\d{10}$/.test(form.phone.trim()))
      newErrors.phone = "Phone number must be exactly 10 digits.";
    return newErrors;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const newErrors = validate();
    setErrors(newErrors);

    if (Object.keys(newErrors).length === 0) {
      setPlaced(true);
      onOrderPlaced(); // tells App to clear cart state
    }
  }

  if (placed) {
    return (
      <p className="confirmation-msg">
         Order placed! Thank you for shopping with TechMart.
      </p>
    );
  }

  return (
    <form onSubmit={handleSubmit} noValidate>
      <label htmlFor="name">Full Name</label>
      <input
        type="text"
        id="name"
        name="name"
        value={form.name}
        onChange={handleChange}
        className={errors.name ? "error-input" : ""}
      />
      <span className="error-msg">{errors.name}</span>
    </form>
  );
}
```

```

<label htmlFor="address">Address</label>
<input
  type="text"
  id="address"
  name="address"
  value={form.address}
  onChange={handleChange}
  className={errors.address ? "error-input" : ""}
/>
<span className="error-msg">{errors.address}</span>

<label htmlFor="city">City</label>
<input
  type="text"
  id="city"
  name="city"
  value={form.city}
  onChange={handleChange}
/>

<label htmlFor="pincode">Pincode</label>
<input
  type="text"
  id="pincode"
  name="pincode"
  maxLength={6}
  value={form.pincode}
  onChange={handleChange}
  className={errors.pincode ? "error-input" : ""}
/>
<span className="error-msg">{errors.pincode}</span>

<label htmlFor="phone">Phone</label>
<input
  type="tel"
  id="phone"
  name="phone"
  maxLength={10}
  value={form.phone}
  onChange={handleChange}
  className={errors.phone ? "error-input" : ""}
/>
<span className="error-msg">{errors.phone}</span>

<label>Payment Method</label>
<div className="payment-options">
  <label>
    <input
      type="radio"
      name="payment"
      value="cod"
      checked={form.payment === "cod"}
      onChange={handleChange}
    />
    Cash on Delivery
  </label>
  <label>
    <input
      type="radio"
      name="payment"
      value="upi"
      checked={form.payment === "upi"}
      onChange={handleChange}
    />
    UPI
  </label>
</div>

```

```
        <input
          type="radio"
          name="payment"
          value="card"
          checked={form.payment === "card"}
          onChange={handleChange}
        />
        Card
      </label>
    </div>

    <button type="submit">Place Order</button>
  </form>
);
}

export default CheckoutForm;
```

PART: 04 TOP-LEVEL STATE — src/App.jsx

Cart state (an array of {product, quantity}) is lifted up and lives here, not in ProductCard (Task 5). addToCart is passed down as a prop. isCartOpen/showCheckout state gives the cart icon + badge its open/close toggle (Task 7).

```
import { useState } from "react";
import ProductList from "../components/ProductList";
import Cart from "../components/Cart";
import CheckoutForm from "../components/CheckoutForm";
import "../index.css";

function App() {
  // Cart state is lifted up here – ProductCard and Cart never own it,
  // they only receive it (or functions to change it) via props.
  const [cart, setCart] = useState([]); // [{ product, quantity }]
  const [isCartOpen, setIsCartOpen] = useState(false);
  const [showCheckout, setShowCheckout] = useState(false);

  function addToCart(product) {
    setCart((prevCart) => {
      const existing = prevCart.find((item) => item.product.id === product.id);
      if (existing) {
        return prevCart.map((item) =>
          item.product.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      return [...prevCart, { product, quantity: 1 }];
    });
    setIsCartOpen(true);
  }

  function removeFromCart(productId) {
    setCart((prevCart) => prevCart.filter((item) => item.product.id !== productId));
  }

  function updateQuantity(productId, newQty) {
    if (newQty < 1 || Number.isNaN(newQty)) newQty = 1;
    setCart((prevCart) =>
      prevCart.map((item) =>
        item.product.id === productId ? { ...item, quantity: newQty } : item
      )
    );
  }

  function handleOrderPlaced() {
    setCart([]);
  }

  const itemCount = cart.reduce((sum, item) => sum + item.quantity, 0);

  return (
    <div className="app">
      <header>
        <div className="navbar">
          <div className="logo">TechMart</div>
          <nav>
            <button
              className="cart-toggle-btn"
              onClick={() => {
                setIsCartOpen((open) => !open);
                setShowCheckout(false);
              }}
            >
              Cart
              {itemCount > 0 && <span className="cart-badge">{itemCount}</span>}
            </button>
          </nav>
        </div>
      </header>
    </div>
  );
}
```

```

        </nav>
      </div>
    </header>

    <main>
      <ProductList onAddToCart={addToCart} />

      {isCartOpen && (
        <div className="cart-drawer">
          {!showCheckout && (
            <Cart
              cart={cart}
              removeFromCart={removeFromCart}
              updateQuantity={updateQuantity}
              onCheckout={() => setShowCheckout(true)}
            />
          )}
          {showCheckout && (
            <div className="checkout-panel">
              <h2>Checkout</h2>
              <CheckoutForm onOrderPlaced={handleOrderPlaced} />
            </div>
          )}
        </div>
      )}
    </main>

    <footer>
      <p>Contact us: support@techmart.com | +91 98765 43210</p>
      <p>&copy; 2026 TechMart. All rights reserved.</p>
    </footer>
  </div>
);
}

export default App;

```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/index.css

```
:root {
  --primary-color: #1f2937;
  --accent-color: #f97316;
  --bg-color: #f9fafb;
  --card-bg: #ffffff;
  --text-muted: #6b7280;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: var(--bg-color);
  color: var(--primary-color);
}
header {
  background: var(--primary-color);
  color: #fff;
  padding: 14px 24px;
}
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.logo { font-size: 1.4rem; font-weight: bold; }
.cart-toggle-btn {
  position: relative;
  background: transparent;
  border: 1px solid #4b5563;
  color: #fff;
  padding: 8px 16px;
  border-radius: 6px;
  cursor: pointer;
  font-size: 1rem;
}
.cart-toggle-btn:hover { border-color: var(--accent-color); }
.cart-badge {
  background: var(--accent-color);
  color: #fff;
  font-size: 0.7rem;
  font-weight: bold;
  border-radius: 999px;
  padding: 1px 7px;
  margin-left: 8px;
}
section h2 { text-align: center; }
.product-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  padding: 30px;
  justify-content: center;
}
.product-card {
  background: var(--card-bg);
  border-radius: 10px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
  padding: 16px;
  width: 220px;
  text-align: center;
}
.product-card img { width: 100%; height: 160px; object-fit: contain; background: #fff; border-radius: 6px; }
.product-card button {
  margin-top: 10px;
  padding: 8px 14px;
}
```

```

background: var(--accent-color);
border: none;
border-radius: 6px;
color: #fff;
cursor: pointer;
}
.product-card button:hover { background: #ea580c; }

.cart-drawer {
max-width: 700px;
margin: 0 auto 30px;
background: var(--card-bg);
border-radius: 10px;
box-shadow: 0 2px 10px rgba(0,0,0,0.15);
padding: 10px 20px 20px;
}
.empty-cart-msg { text-align: center; color: var(--text-muted); padding: 20px; }
table { width: 100%; border-collapse: collapse; margin-top: 10px; }
table th, table td {
border: 1px solid #e5e7eb;
padding: 8px;
text-align: center;
}
table th { background: var(--primary-color); color: #fff; }
.total-row td { font-weight: bold; background: #f3f4f6; }
.qty-input { width: 55px; padding: 4px; text-align: center; }
.remove-btn {
background: #dc2626;
border: none;
color: #fff;
padding: 6px 12px;
border-radius: 6px;
cursor: pointer;
}
.remove-btn:hover { background: #b91c1c; }
.checkout-btn {
margin-top: 16px;
padding: 10px 20px;
background: var(--accent-color);
border: none;
border-radius: 6px;
color: #fff;
cursor: pointer;
}
.checkout-panel form {
max-width: 100%;
}
form label { display: block; margin-top: 12px; margin-bottom: 4px; }
form input[type="text"],
form input[type="tel"] {
width: 100%;
padding: 8px;
border: 1px solid #d1d5db;
border-radius: 6px;
}
form input.error-input { border-color: #dc2626; }
.error-msg {
color: #dc2626;
font-size: 0.82rem;
display: block;
margin-top: 4px;
min-height: 1em;
}
.payment-options label { display: inline-block; margin-right: 16px; font-weight: normal; }
form button {
margin-top: 20px;
padding: 10px 20px;
background: var(--accent-color);

```

```
border: none;
border-radius: 6px;
color: #fff;
cursor: pointer;
}
form button:hover { background: #ea580c; }
.confirmation-msg {
  text-align: center;
  background: #dcfce7;
  color: #166534;
  border: 1px solid #86efac;
  padding: 16px;
  border-radius: 8px;
  font-weight: bold;
}
footer {
  background: var(--primary-color);
  color: #d1d5db;
  text-align: center;
  padding: 16px;
  margin-top: 30px;
}
```

OUTPUT SCREENSHOTS

ProductList + ProductCard — initial render

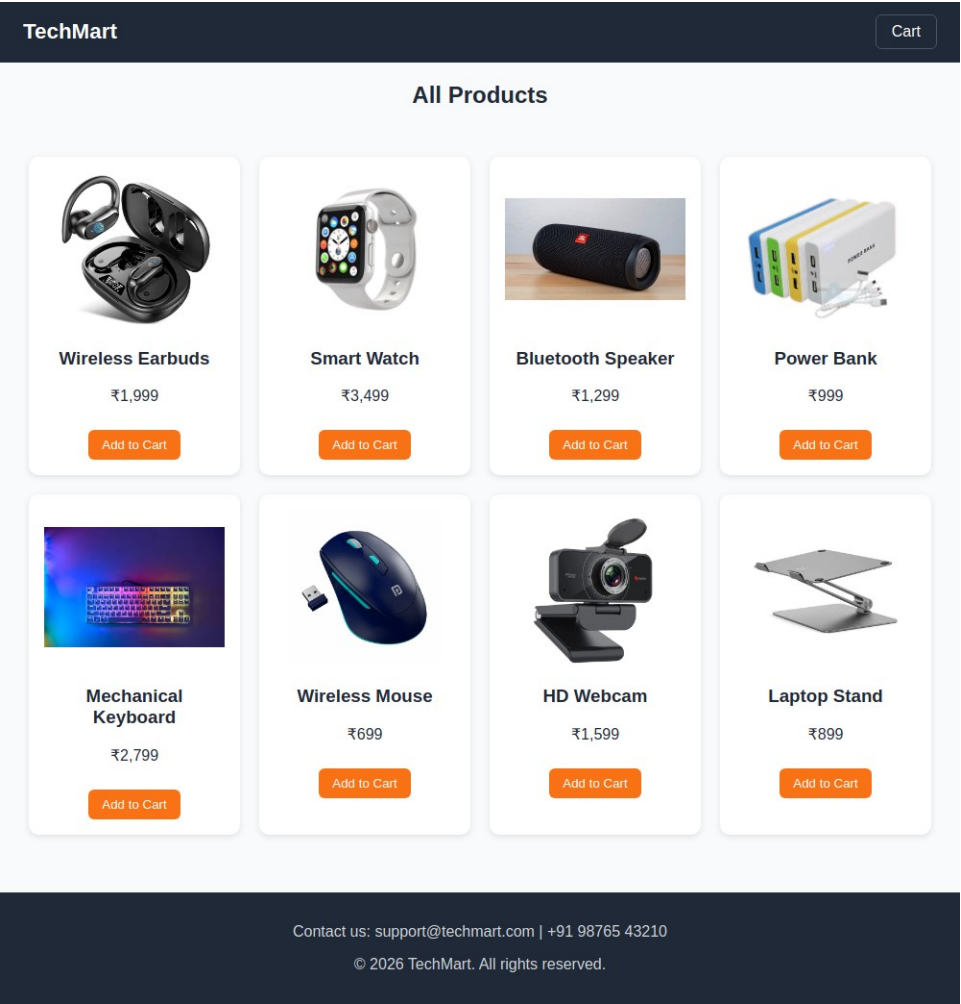


Fig 1: All 8 products rendered by `ProductList.map()`, each as a `ProductCard` component keyed by `product.id`.

Cart opens after Add to Cart — state lifted to App

TechMart

Cart 4

All Products



Wireless Earbuds

₹1,999

Add to Cart



Smart Watch

₹3,499

Add to Cart



Bluetooth Speaker

₹1,299

Add to Cart



Power Bank

₹999

Add to Cart



Mechanical Keyboard

₹2,799

Add to Cart



Wireless Mouse

₹699

Add to Cart



HD Webcam

₹1,599

Add to Cart



Laptop Stand

₹899

Add to Cart

Your Cart

Item	Quantity	Price	Subtotal	
Wireless Earbuds	2	₹1,999	₹3,998	<button>Remove</button>
Smart Watch	1	₹3,499	₹3,499	<button>Remove</button>
Power Bank	1	₹999	₹999	<button>Remove</button>
Grand Total			₹8,496	

Proceed to Checkout

Contact us: support@techmart.com | +91 98765 43210

© 2026 TechMart. All rights reserved.

Fig 2: Clicking Add to Cart on ProductCard calls the onAddToCart prop, which updates cart state in App. Navbar badge reads item count from that same state (Task 5 & 7).

Quantity change and item removal

TechMart

Cart 5

All Products



Wireless Earbuds

₹1,999

Add to Cart



Smart Watch

₹3,499

Add to Cart



Bluetooth Speaker

₹1,299

Add to Cart



Power Bank

₹999

Add to Cart



Mechanical Keyboard

₹2,799

Add to Cart



Wireless Mouse

₹699

Add to Cart



HD Webcam

₹1,599

Add to Cart



Laptop Stand

₹899

Add to Cart

Your Cart

Item	Quantity	Price	Subtotal	
Wireless Earbuds	3	₹1,999	₹5,997	<button>Remove</button>
Smart Watch	1	₹3,499	₹3,499	<button>Remove</button>
Power Bank	1	₹999	₹999	<button>Remove</button>
Grand Total			₹10,495	

Proceed to Checkout

Contact us: support@techmart.com | +91 98765 43210

© 2026 TechMart. All rights reserved.

Fig 3: Editing the quantity input calls `updateQuantity(id, value)`, a prop passed down from App; Grand Total recalculates immediately.

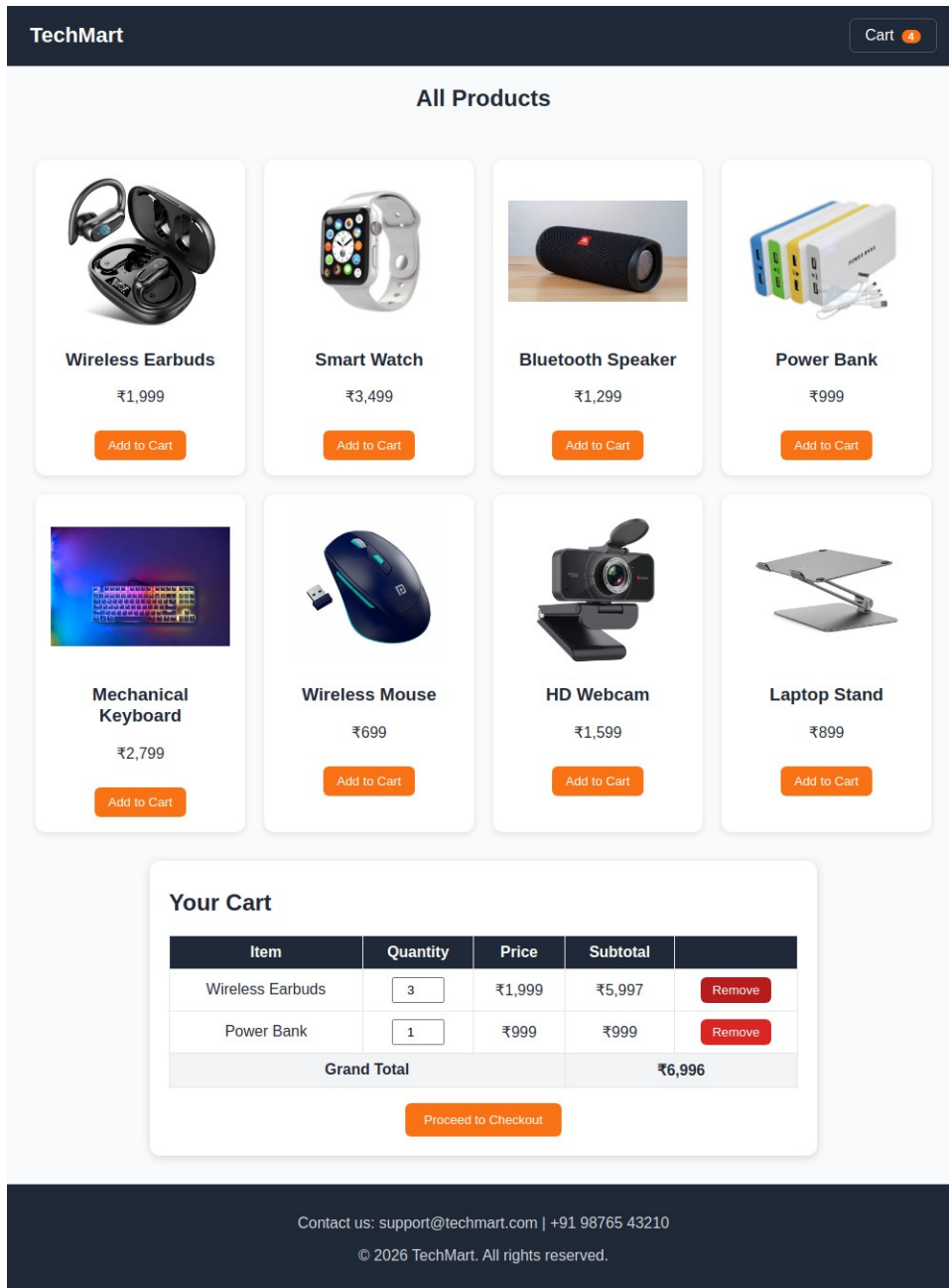


Fig 4: Smart Watch removed via the Remove button (removeFromCart prop); badge and grand total update accordingly.

CheckoutForm — controlled inputs with validation

TechMart

Cart 4

All Products



Wireless Earbuds

₹1,999

Add to Cart



Smart Watch

₹3,499

Add to Cart



Bluetooth Speaker

₹1,299

Add to Cart



Power Bank

₹999

Add to Cart



Mechanical Keyboard

₹2,799

Add to Cart



Wireless Mouse

₹699

Add to Cart



HD Webcam

₹1,599

Add to Cart



Laptop Stand

₹899

Add to Cart

Checkout

Full Name

Name is required.

Address

Address is required.

City

Pincode

12

Pincode must be exactly 6 digits.

Phone

999

Phone number must be exactly 10 digits.

Payment Method

☒ Cash on Delivery ☐ UPI ☐ Card

Place Order

Contact us: support@techmart.com | +91 98765 43210

© 2026 TechMart. All rights reserved.

Fig 5: Empty Name/Address, a 2-digit Pincode and a 3-digit Phone are all rejected with inline error text, sourced from the errors state object — no alert() used.

Successful order placement

TechMart

Cart

All Products



Wireless Earbuds

₹1,999

Add to Cart



Smart Watch

₹3,499

Add to Cart



Bluetooth Speaker

₹1,299

Add to Cart



Power Bank

₹999

Add to Cart



Mechanical Keyboard

₹2,799

Add to Cart



Wireless Mouse

₹699

Add to Cart



HD Webcam

₹1,599

Add to Cart



Laptop Stand

₹899

Add to Cart

Checkout

✔ Order placed! Thank you for shopping with TechMart.

Contact us: support@techmart.com | +91 98765 43210

© 2026 TechMart. All rights reserved.

Fig 6: Valid data submitted → onOrderPlaced() fires, clearing cart state in App and showing the confirmation message.

Conditional rendering — empty cart message (Task 9)

TechMart

Cart

All Products



Wireless Earbuds

₹1,999

Add to Cart



Smart Watch

₹3,499

Add to Cart



Bluetooth Speaker

₹1,299

Add to Cart



Power Bank

₹999

Add to Cart



Mechanical Keyboard

₹2,799

Add to Cart



Wireless Mouse

₹699

Add to Cart



HD Webcam

₹1,599

Add to Cart



Laptop Stand

₹899

Add to Cart

Your Cart

Your cart is empty.

Contact us: support@techmart.com | +91 98765 43210

© 2026 TechMart. All rights reserved.

Fig 7: Reopening the cart after checkout shows the "Your cart is empty" message via Cart's early-return conditional render, instead of an empty table.